КОМПОЗИЦИЯ. ПЕРЕЗАГРУЗКА ОПЕРАТОРОВ

Цель работы

Изучить и практически освоить принципы композиции объектов и перегрузки операторов в объектно-ориентированном программировании на Python, сформировать навыки проектирования классов с использованием отношений "has-a" (композиция) и создания интуитивно понятных интерфейсов через переопределение стандартных операторов.

Задание № 1

Требуется написать программу и разработать интерфейс, которая вычисляет площадь для определенных работ в помещении. При этом есть части, в которых работы проводить не надо. Сохранять саму площадь помещения в поле не обязательно. Площади должны вычислять лишь при необходимости путем вызова методов. Программа должна запрашивать у пользователя данные и выдает ему площадь работ и количество необходимых материалов.

Дано первоначальное описание, но его требуется исправить и доработать, согласно плану.

Варианты к заданию № 1

1. **Расчет площади покраски стен**

Требуется написать программу, которая вычисляет площадь покраски стен помещения. При этом окна, двери, плинтусы и розетки красить не надо.

```
class Opening:
    def __init__(self, width, height):
        self.area = width * height

class Room:
    def __init__(self, length, width, height):
        self.wall_area = 2 * height * (length + width)
        self.openings = []
    def add_opening(self, w, h):
        self.openings.append(Opening(w, h))
```

```
    def paint_area(self):
        area = self.wall_area
        for i in self.openings:
            area -= i.area
        return area
```

Исправьте код так, чтобы у объектов Room были поля – length, width, height и openings. Площади (полная и для покраски) должны вычисляться лишь при необходимости. Добавьте метод, который принимает расход краски (м²/л) и возвращает количество необходимой краски.

Разработайте интерфейс программы.

2. **Расчет напольного покрытия**

Требуется написать программу, которая вычисляет площадь укладки напольного покрытия. При этом колонны и выступы покрывать не надо.

```
class Obstacle:
    def __init__(self, x, y):
        self.area = x * y

class Floor:
    def __init__(self, x, y):
        self.total_area = x * y
        self.obstacles = []
    def add_obstacle(self, w, h):
        self.obstacles.append(Obstacle(w, h))
    def work_area(self):
        area = self.total_area
        for i in self.obstacles:
            area -= i.area
        return area
```

Исправьте код так, чтобы у объектов Floor были поля – length, width и obstacles.

Добавьте метод, который принимает размеры одной плитки и возвращает количество необходимых плиток.

Разработайте интерфейс программы.

3. **Расчет кровельного материала**

Требуется написать программу, которая вычисляет площадь покрытия крыши. При этом мансардные окна и вентиляционные выходы покрывать не надо.

```
class RoofOpening:
    def __init__(self, x, y):
        self.area = x * y

class Roof:
    def __init__(self, length, width, slope):
        self.area = length * width * slope
        self.openings = []
    def add_opening(self, w, h):
        self.openings.append(RoofOpening(w, h))
    def cover_area(self):
        area = self.area
        for i in self.openings:
            area -= i.area
        return area
```

Исправьте код так, чтобы у объектов Roof были поля – length, width, slope и openings. Добавьте метод, который принимает размеры одного листа кровли и возвращает количество листов.

Разработайте интерфейс программы.

4. **Расчет потолочных панелей**

Требуется написать программу, которая вычисляет площадь потолка для монтажа панелей. При этом светильники и вентиляционные решетки не закрывать панелями.

```
class CeilingElement:
    def __init__(self, x, y):
        self.area = x * y

class Ceiling:
    def __init__(self, x, y):
        self.total_area = x * y
        self.elements = []
    def add_element(self, w, h):
        self.elements.append(CeilingElement(w, h))
```

```
    def panel_area(self):
        area = self.total_area
        for i in self.elements:
            area -= i.area
        return area
```

Исправьте код так, чтобы у объектов Ceiling были поля – length, width и elements. Добавьте метод, который принимает размеры одной панели и возвращает количество панелей.

Разработайте интерфейс программы.

5. **Расчет плитки для ванной**

Требуется написать программу, которая вычисляет площадь укладки плитки в ванной. При этом дверь, окно и сантехнику не плитковать.

```
class Exclusion:
    def __init__(self, x, y):
        self.area = x * y

class Bathroom:
    def __init__(self, wall1, wall2, height):
        self.wall_area = 2 * height * (wall1 + wall2)
        self.exclusions = []
    def add_exclusion(self, w, h):
        self.exclusions.append(Exclusion(w, h))
    def tile_area(self):
        area = self.wall_area
        for i in self.exclusions:
            area -= i.area
        return area
```

Исправьте код так, чтобы у объектов Bathroom были поля – length, width, height и exclusions. Добавьте метод, который принимает размеры одной плитки и возвращает количество плиток.

Разработайте интерфейс программы.

6. **Расчет линолеума**

Требуется написать программу, которая вычисляет площадь укладки линолеума. При этом выступы и ниши не покрывать.

```
class Cutout:
    def __init__(self, x, y):
        self.area = x * y

class RoomFloor:
    def __init__(self, x, y):
        self.total_area = x * y
        self.cutouts = []
    def add_cutout(self, w, h):
        self.cutouts.append(Cutout(w, h))
    def covering_area(self):
        area = self.total_area
        for i in self.cutouts:
            area -= i.area
        return area
```

Исправьте код так, чтобы у объектов RoomFloor были поля – length, width и cutouts. Добавьте метод, который принимает ширину рулона линолеума и возвращает длину отреза.

Разработайте интерфейс программы.

7. **Расчет коврового покрытия**

Требуется написать программу, которая вычисляет площадь укладки коврового покрытия. При этом камин и встроенную мебель не покрывать.

```
class FixedElement:
    def __init__(self, x, y):
        self.area = x * y

class LivingRoom:
    def __init__(self, x, y):
        self.floor_area = x * y
        self.fixed_elements = []
    def add_element(self, w, h):
        self.fixed_elements.append(FixedElement(w, h))
```

```
    def carpet_area(self):
        area = self.floor_area
        for i in self.fixed_elements:
            area -= i.area
        return area
```

Исправьте код так, чтобы у объектов LivingRoom были поля – length, width и fixed_elements. Добавьте метод, который принимает стандартные размеры ковра и предлагает варианты раскроя.

Разработайте интерфейс программы.

8. **Расчет гипсокартона**

Требуется написать программу, которая вычисляет площадь обшивки гипсокартоном. При этом проемы и коммуникационные отверстия не закрывать.

```
class Opening:
    def __init__(self, x, y):
        self.area = x * y

class Wall:
    def __init__(self, x, y):
        self.total_area = x * y
        self.openings = []
    def add_opening(self, w, h):
        self.openings.append(Opening(w, h))
    def drywall_area(self):
        area = self.total_area
        for i in self.openings:
            area -= i.area
        return area
```

Исправьте код так, чтобы у объектов Wall были поля – length, height и openings. Добавьте метод, который принимает размеры листа гипсокартона и возвращает количество листов.

Разработайте интерфейс программы.

9. **Расчет сайдинга для фасада**

Требуется написать программу, которая вычисляет площадь обшивки фасада сайдингом. При этом окна и двери не обшивать.

```
class FacadeOpening:
    def __init__(self, x, y):
        self.area = x * y

class Facade:
    def __init__(self, width, height):
        self.total_area = width * height
        self.openings = []
    def add_opening(self, w, h):
        self.openings.append(FacadeOpening(w, h))
    def siding_area(self):
        area = self.total_area
        for i in self.openings:
            area -= i.area
        return area
```

Исправьте код так, чтобы у объектов Facade были поля – width, height и openings. Добавьте метод, который принимает размеры одной панели сайдинга и возвращает количество панелей.

Разработайте интерфейс программы.

10. **Расчет террасной доски**

Требуется написать программу, которая вычисляет площадь укладки террасной доски. При этом ограждения и столбы не учитывать в площади.

```
class Post:
    def __init__(self, x, y):
        self.area = x * y

class Terrace:
    def __init__(self, x, y):
        self.total_area = x * y
        self.posts = []
    def add_post(self, w, h):
        self.posts.append(Post(w, h))
```

```
    def decking_area(self):
        area = self.total_area
        for i in self.posts:
            area -= i.area
        return area
```

Исправьте код так, чтобы у объектов Terrace были поля – length, width и posts. Добавьте метод, который принимает размеры одной доски и возвращает количество досок.

Разработайте интерфейс программы.

11. **Расчет теплоизоляции**

Требуется написать программу, которая вычисляет площадь утепления стен. При этом оконные и дверные проемы не утеплять.

```
class InsulationExclusion:
    def __init__(self, x, y):
        self.area = x * y

class Insulation:
    def __init__(self, perimeter, height):
        self.total_area = perimeter * height
        self.exclusions = []
    def add_exclusion(self, w, h):
        self.exclusions.append(InsulationExclusion(w, h))
    def insulation_area(self):
        area = self.total_area
        for i in self.exclusions:
            area -= i.area
        return area
```

Исправьте код так, чтобы у объектов Insulation были поля – perimeter, height и exclusions. Добавьте метод, который принимает размеры рулона утеплителя и возвращает количество рулонов.

Разработайте интерфейс программы.

12. **Расчет гидроизоляции**

Требуется написать программу, которая вычисляет площадь гидроизоляции подвала. При этом вводы коммуникаций не учитывать.

```
class Penetration:
    def __init__(self, diameter):
        self.area = 3.14 * (diameter/2) ** 2

class Basement:
    def __init__(self, length, width, height):
        self.total_area = 2 * height * (length + width) + length * width
        self.penetrations = []
    def add_penetration(self, d):
        self.penetrations.append(Penetration(d))
    def waterproofing_area(self):
        area = self.total_area
        for i in self.penetrations:
            area -= i.area
        return area
```

Исправьте код так, чтобы у объектов Basement были поля – length, width, height и penetrations. Добавьте метод, который принимает размеры рулона гидроизоляции и возвращает количество рулонов.

Разработайте интерфейс программы.

13. **Расчет штукатурки**

Требуется написать программу, которая вычисляет площадь оштукатуривания стен. При этом дверные и оконные проемы не штукатурить.

```
class PlasterExclusion:
    def __init__(self, x, y):
        self.area = x * y

class PlasterArea:
    def __init__(self, walls_area):
        self.total_area = walls_area
        self.exclusions = []
    def add_exclusion(self, w, h):
        self.exclusions.append(PlasterExclusion(w, h))
```

```
    def plaster_area(self):
        area = self.total_area
        for i in self.exclusions:
            area -= i.area
        return area
```

Исправьте код так, чтобы у объектов PlasterArea были поля – walls_area и exclusions. Добавьте метод, который принимает расход штукатурки и возвращает необходимое количество смеси.

Разработайте интерфейс программы.

14. **Расчет ламината**

Требуется написать программу, которая вычисляет площадь укладки ламината. При этом дверные проемы и выступы не учитывать.

```
class LaminateExclusion:
    def __init__(self, x, y):
        self.area = x * y

class LaminateFloor:
    def __init__(self, x, y):
        self.total_area = x * y
        self.exclusions = []
    def add_exclusion(self, w, h):
        self.exclusions.append(LaminateExclusion(w, h))
    def laminate_area(self):
        area = self.total_area
        for i in self.exclusions:
            area -= i.area
        return area
```

Исправьте код так, чтобы у объектов LaminateFloor были поля – length, width и exclusions. Добавьте метод, который принимает размеры одной доски ламината и возвращает количество досок.

Разработайте интерфейс программы.

15. **Расчет потолочной плитки**

Требуется написать программу, которая вычисляет площадь потолка для монтажа плитки. При этом светильники и вентиляционные решетки не учитывать.

```
class CeilingExclusion:
    def __init__(self, x, y):
        self.area = x * y

class CeilingTile:
    def __init__(self, x, y):
        self.total_area = x * y
        self.exclusions = []
    def add_exclusion(self, w, h):
        self.exclusions.append(CeilingExclusion(w, h))
    def tile_area(self):
        area = self.total_area
        for i in self.exclusions:
            area -= i.area
        return area
```

Исправьте код так, чтобы у объектов CeilingTile были поля – length, width и exclusions. Добавьте метод, который принимает размеры одной плитки и возвращает количество плиток.

Разработайте интерфейс программы.

16. **Расчет паркетной доски**

Требуется написать программу, которая вычисляет площадь укладки паркетной доски. При этом камины и колонны не учитывать.

```
class ParquetExclusion:
    def __init__(self, x, y):
        self.area = x * y

class Parquet:
    def __init__(self, x, y):
        self.total_area = x * y
        self.exclusions = []
    def add_exclusion(self, w, h):
        self.exclusions.append(ParquetExclusion(w, h))
```

```
    def parquet_area(self):
        area = self.total_area
        for i in self.exclusions:
            area -= i.area
        return area
```

Исправьте код так, чтобы у объектов Parquet были поля – length, width и exclusions. Добавьте метод, который принимает размеры одной доски и возвращает количество досок.

Разработайте интерфейс программы.

17. **Расчет керамогранита**

Требуется написать программу, которая вычисляет площадь укладки керамогранита. При этом сантехнические отверстия не учитывать.

```
class TileExclusion:
    def __init__(self, x, y):
        self.area = x * y

class PorcelainTile:
    def __init__(self, x, y):
        self.total_area = x * y
        self.exclusions = []
    def add_exclusion(self, w, h):
        self.exclusions.append(TileExclusion(w, h))
    def porcelain_area(self):
        area = self.total_area
        for i in self.exclusions:
            area -= i.area
        return area
```

Исправьте код так, чтобы у объектов PorcelainTile были поля – length, width и exclusions. Добавьте метод, который принимает размеры одной плитки и возвращает количество плиток.

Разработайте интерфейс программы.

18. **Расчет мозаики**

Требуется написать программу, которая вычисляет площадь укладки мозаики. При этом декоративные вставки не учитывать.

```
class MosaicExclusion:
    def __init__(self, x, y):
        self.area = x * y

class Mosaic:
    def __init__(self, x, y):
        self.total_area = x * y
        self.exclusions = []
    def add_exclusion(self, w, h):
        self.exclusions.append(MosaicExclusion(w, h))
    def mosaic_area(self):
        area = self.total_area
        for i in self.exclusions:
            area -= i.area
        return area
```

Исправьте код так, чтобы у объектов Mosaic были поля – length, width и exclusions. Добавьте метод, который принимает размеры одного листа мозаики и возвращает количество листов.

Разработайте интерфейс программы.

19. **Расчет искусственного газона**

Требуется написать программу, которая вычисляет площадь укладки искусственного газона. При этом клумбы и дорожки не покрывать.

```
class LawnExclusion:
    def __init__(self, x, y):
        self.area = x * y

class ArtificialLawn:
    def __init__(self, x, y):
        self.total_area = x * y
        self.exclusions = []
    def add_exclusion(self, w, h):
        self.exclusions.append(LawnExclusion(w, h))
```

```
    def lawn_area(self):
        area = self.total_area
        for i in self.exclusions:
            area -= i.area
        return area
```

Исправьте код так, чтобы у объектов ArtificialLawn были поля – length, width и exclusions. Добавьте метод, который принимает ширину рулона газона и возвращает длину отреза.

Разработайте интерфейс программы.

20. **Расчет натяжного потолка**

Требуется написать программу, которая вычисляет площадь натяжного потолка. При этом встроенные светильники не учитывать.

```
class LightOpening:
    def __init__(self, diameter):
        self.area = 3.14 * (diameter/2) ** 2

class StretchCeiling:
    def __init__(self, x, y):
        self.total_area = x * y
        self.lights = []
    def add_light(self, d):
        self.lights.append(LightOpening(d))
    def ceiling_area(self):
        area = self.total_area
        for i in self.lights:
            area -= i.area
        return area
```

Исправьте код так, чтобы у объектов StretchCeiling были поля – length, width и lights. Добавьте метод, который принимает стандартные размеры полотна и предлагает варианты раскроя.

Разработайте интерфейс программы.

21. **Расчет виниловых обоев**

Требуется написать программу, которая вычисляет площадь оклейки виниловыми обоями. При этом оконные и дверные проемы не оклеивать.

```
class VinylExclusion:
    def __init__(self, x, y):
        self.area = x * y

class VinylWall:
    def __init__(self, perimeter, height):
        self.total_area = perimeter * height
        self.exclusions = []
    def add_exclusion(self, w, h):
        self.exclusions.append(VinylExclusion(w, h))
    def vinyl_area(self):
        area = self.total_area
        for i in self.exclusions:
            area -= i.area
        return area
```

Исправьте код так, чтобы у объектов VinylWall были поля – perimeter, height и exclusions. Добавьте метод, который принимает размеры рулона и возвращает количество рулонов.

Разработайте интерфейс программы.

22. **Расчет декоративного камня**

Требуется написать программу, которая вычисляет площадь укладки декоративного камня. При этом оконные и дверные проемы не облицовывать.

```
class StoneExclusion:
    def __init__(self, x, y):
        self.area = x * y

class DecorativeStone:
    def __init__(self, length, height):
        self.total_area = length * height
        self.exclusions = []
    def add_exclusion(self, w, h):
        self.exclusions.append(StoneExclusion(w, h))
```

```
    def stone_area(self):
        area = self.total_area
        for i in self.exclusions:
            area -= i.area
        return area
```

Исправьте код так, чтобы у объектов DecorativeStone были поля – length, height и exclusions. Добавьте метод, который принимает размеры одного камня и возвращает количество камней.

Разработайте интерфейс программы.

23. **Расчет клинкерной плитки**

Требуется написать программу, которая вычисляет площадь укладки клинкерной плитки. При этом угловые элементы и подрезки учитывать отдельно.

```
class ClinkerExclusion:
    def __init__(self, x, y):
        self.area = x * y

class ClinkerTile:
    def __init__(self, x, y):
        self.total_area = x * y
        self.exclusions = []
    def add_exclusion(self, w, h):
        self.exclusions.append(ClinkerExclusion(w, h))
    def clinker_area(self):
        area = self.total_area
        for i in self.exclusions:
            area -= i.area
        return area
```

Исправьте код так, чтобы у объектов ClinkerTile были поля – length, width и exclusions. Добавьте метод, который принимает размеры плитки и возвращает количество с учетом подрезки.

Разработайте интерфейс программы.

24. **Расчет резинового покрытия**

Требуется написать программу, которая вычисляет площадь укладки резинового покрытия. При этом дренажные отверстия не учитывать.

```
class RubberExclusion:
    def __init__(self, x, y):
        self.area = x * y

class RubberFloor:
    def __init__(self, x, y):
        self.total_area = x * y
        self.exclusions = []
    def add_exclusion(self, w, h):
        self.exclusions.append(RubberExclusion(w, h))
    def rubber_area(self):
        area = self.total_area
        for i in self.exclusions:
            area -= i.area
        return area
```

Исправьте код так, чтобы у объектов RubberFloor были поля – length, width и exclusions. Добавьте метод, который принимает размеры плитки и возвращает количество плиток.

Разработайте интерфейс программы.

25. **Расчет акустических панелей**

Требуется написать программу, которая вычисляет площадь монтажа акустических панелей. При этом встроенные динамики и системы вентиляции не закрывать.

```
class AcousticExclusion:
    def __init__(self, x, y):
        self.area = x * y

class AcousticWall:
    def __init__(self, length, height):
        self.total_area = length * height
        self.exclusions = []
    def add_exclusion(self, w, h):
        self.exclusions.append(AcousticExclusion(w, h))
```

```
    def acoustic_area(self):
        area = self.total_area
        for i in self.exclusions:
            area -= i.area
        return area
```

Исправьте код так, чтобы у объектов AcousticWall были поля – length, height и exclusions. Добавьте метод, который принимает размеры панели и возвращает количество панелей.

Разработайте интерфейс программы.

Задание № 2

Напишите программу по следующему описанию.

Рекомендации:

– проверка типов: всегда проверяйте тип other в перегруженных операторах;

– обработка граничных значений: особенно важно для классов с ограниченным диапазоном (Brightness);

– округление: в методах __truediv__ используйте округление до целого, если требуется;

– форматирование вывода: следите за читаемостью выводимых строк;

– валидация данных: проверяйте корректность входных данных, особенно при вызове объекта.

Варианты к заданию № 2

Вариант 1: Класс Money (Деньги)

Напишите класс Money по следующему описанию. В конструкторе класса инициируется поле, содержащее количество денег, выраженное вещественным числом.

Класс включает методы перегрузки арифметических операторов:

__add__ – сложение, __sub__ – вычитание, __mul__ – умножение, __truediv__ – деление.

В классе код этих методов должен выполнять арифметические операции с количеством денег.

Метод __truediv__ перегружает обычное / деление.

Класс включает метод convert, который помимо self принимает код валюты и курс обмена, и возвращает строку вида "100.00 USD = 7500.00 RUB", где количество денег пересчитано по курсу.

Вызов экземпляра класса Money в нотации функции с одним аргументом должен приводить к перезаписи значения поля, в котором хранится количество денег, на переданное значение.

Вариант 2: Класс Book (Книга)

Напишите класс Book по следующему описанию.

В конструкторе класса инициируется поле, содержащее количество страниц, выраженное целым числом.

Класс включает методы перегрузки арифметических операторов:

__add__ – сложение, __sub__ – вычитание, __mul__ – умножение, __truediv__ – деление.

В классе код этих методов должен выполнять арифметические операции с количеством страниц.

Метод __truediv__ перегружает обычное / деление с округлением до целого.

Класс включает метод reading_time, который помимо self принимает скорость чтения (страниц в час) и возвращает строку вида "Для прочтения 300 страниц потребуется 10 часов", где время вычисляется исходя из количества страниц и скорости.

Вызов экземпляра класса Book в нотации функции с одним аргументом должен приводить к перезаписи значения поля, в котором хранится количество страниц, на переданное значение.

Вариант 3: Класс Temperature (Температура)

Напишите класс Temperature по следующему описанию.

В конструкторе класса инициируется поле, содержащее значение температуры в Цельсиях, выраженное вещественным числом.

Класс включает методы перегрузки арифметических операторов:

__add__ – сложение, __sub__ – вычитание, __mul__ – умножение, __truediv__ – деление.

В классе код этих методов должен выполнять арифметические операции с температурой.

Класс включает метод convert, который помимо self принимает шкалу ('F' для Фаренгейта, 'K' для Кельвина)

и возвращает строку вида "25.0 C = 77.0 F", где температура переведена в указанную шкалу.

Вызов экземпляра класса Temperature в нотации функции с одним аргументом должен приводить к перезаписи значения поля, в котором хранится температура, на переданное значение.

Вариант 4: Класс Distance (Расстояние)

Напишите класс Distance по следующему описанию.

В конструкторе класса инициируется поле, содержащее расстояние в метрах, выраженное вещественным числом.

Класс включает методы перегрузки арифметических операторов:

__add__ – сложение, __sub__ – вычитание, __mul__ – умножение, __truediv__ – деление.

В классе код этих методов должен выполнять арифметические операции с расстоянием.

Класс включает метод convert, который помимо self принимает единицу измерения ('km', 'mile', 'foot') и возвращает строку вида "1000 m = 1.0 km", где расстояние переведено в указанные единицы.

Вызов экземпляра класса Distance в нотации функции с одним аргументом должен приводить к перезаписи значения поля, в котором хранится расстояние, на переданное значение.

Вариант 5: Класс Time (Время)

Напишите класс Time по следующему описанию.

В конструкторе класса инициируется поле, содержащее время в секундах, выраженное целым числом.

Класс включает методы перегрузки арифметических операторов:

__add__ – сложение, __sub__ – вычитание, __mul__ – умножение, __truediv__ – деление.

В классе код этих методов должен выполнять арифметические операции со временем в секундах.

Метод __truediv__ перегружает обычное / деление с округлением до целого.

Класс включает метод format, который помимо self возвращает строку вида "2:30:45", где время представлено в формате часы:минуты:секунды.

Вызов экземпляра класса Time в нотации функции с одним аргументом должен приводить к перезаписи значения поля, в котором хранится время, на переданное значение.

Вариант 6: Класс Weight (Вес)

Напишите класс Weight по следующему описанию.

В конструкторе класса инициируется поле, содержащее вес в килограммах, выраженное вещественным числом.

Класс включает методы перегрузки арифметических операторов:

__add__ – сложение, __sub__ – вычитание, __mul__ – умножение, __truediv__ – деление.

В классе код этих методов должен выполнять арифметические операции с весом.

Класс включает метод convert, который помимо self принимает единицу измерения ('g', 'lb', 'oz') и возвращает строку вида "1.0 kg = 1000.0 g", где вес переведен в указанные единицы.

Вызов экземпляра класса Weight в нотации функции с одним аргументом должен приводить к перезаписи значения поля, в котором хранится вес, на переданное значение.

Вариант 7: Класс Volume (Объем)

Напишите класс Volume по следующему описанию.

В конструкторе класса инициируется поле, содержащее объем в литрах, выраженное вещественным числом.

Класс включает методы перегрузки арифметических операторов:

__add__ – сложение, __sub__ – вычитание, __mul__ – умножение, __truediv__ – деление.

В классе код этих методов должен выполнять арифметические операции с объемом.

Класс включает метод convert, который помимо self принимает единицу измерения ('ml', 'gal', 'm3') и возвращает строку вида "1.0 L = 1000.0 ml", где объем переведен в указанные единицы.

Вызов экземпляра класса Volume в нотации функции с одним аргументом должен приводить к перезаписи значения поля, в котором хранится объем, на переданное значение.

Вариант 8: Класс Speed (Скорость)

Напишите класс Speed по следующему описанию.

В конструкторе класса инициируется поле, содержащее скорость в м/с, выраженное вещественным числом.

Класс включает методы перегрузки арифметических операторов:

__add__ – сложение, __sub__ – вычитание, __mul__ – умножение, __truediv__ – деление.

В классе код этих методов должен выполнять арифметические операции со скоростью.

Класс включает метод convert, который помимо self принимает единицу измерения ('km/h', 'mph', 'knot') и возвращает строку вида "10.0 m/s = 36.0 km/h", где скорость переведена в указанные единицы.

Вызов экземпляра класса Speed в нотации функции с одним аргументом должен приводить к перезаписи значения поля, в котором хранится скорость, на переданное значение.

Вариант 9: Класс Area (Площадь)

Напишите класс Area по следующему описанию.

В конструкторе класса инициируется поле, содержащее площадь в квадратных метрах, выраженное вещественным числом.

Класс включает методы перегрузки арифметических операторов:

__add__ – сложение, __sub__ – вычитание, __mul__ – умножение, __truediv__ – деление.

В классе код этих методов должен выполнять арифметические операции с площадью.

Класс включает метод convert, который помимо self принимает единицу измерения ('ha', 'acre', 'km2') и возвращает строку вида "10000.0 m2 = 1.0 ha", где площадь переведена в указанные единицы.

Вызов экземпляра класса Area в нотации функции с одним аргументом должен приводить к перезаписи значения поля, в котором хранится площадь, на переданное значение.

Вариант 10: Класс Energy (Энергия)

Напишите класс Energy по следующему описанию.

В конструкторе класса инициируется поле, содержащее энергию в джоулях, выраженное вещественным числом.

Класс включает методы перегрузки арифметических операторов:

__add__ – сложение, __sub__ – вычитание, __mul__ – умножение, __truediv__ – деление.

В классе код этих методов должен выполнять арифметические операции с энергией.

Класс включает метод convert, который помимо self принимает единицу измерения ('cal', 'kWh', 'eV') и возвращает строку вида "4184.0 J = 1000.0 cal", где энергия переведена в указанные единицы.

Вызов экземпляра класса Energy в нотации функции с одним аргументом должен приводить к перезаписи значения поля, в котором хранится энергия, на переданное значение.

Вариант 11: Класс DataSize (Размер данных)

Напишите класс DataSize по следующему описанию.

В конструкторе класса инициируется поле, содержащее размер данных в байтах, выраженное целым числом.

Класс включает методы перегрузки арифметических операторов:

__add__ – сложение, __sub__ – вычитание, __mul__ – умножение, __truediv__ – деление.

В классе код этих методов должен выполнять арифметические операции с размером данных.

Метод __truediv__ перегружает обычное / деление с округлением до целого.

Класс включает метод format_size, который помимо self возвращает строку вида "1.5 MB", где размер автоматически конвертируется в подходящие единицы (B, KB, MB, GB, TB).

Вызов экземпляра класса DataSize в нотации функции с одним аргументом должен приводить к перезаписи значения поля, в котором хранится размер данных, на переданное значение.

Вариант 12: Класс Angle (Угол)

Напишите класс Angle по следующему описанию.

В конструкторе класса инициируется поле, содержащее угол в градусах, выраженное вещественным числом.

Класс включает методы перегрузки арифметических операторов:

__add__ – сложение, __sub__ – вычитание, __mul__ – умножение, __truediv__ – деление.

В классе код этих методов должен выполнять арифметические операции с углом.

Класс включает метод convert, который помимо self принимает единицу измерения ('rad', 'grad') и возвращает строку вида "180.0° = 3.14 rad", где угол переведен в указанные единицы.

Вызов экземпляра класса Angle в нотации функции с одним аргументом должен приводить к перезаписи значения поля, в котором хранится угол, на переданное значение.

Вариант 13: Класс Pressure (Давление)

Напишите класс Pressure по следующему описанию.

В конструкторе класса инициируется поле, содержащее давление в паскалях, выраженное вещественным числом.

Класс включает методы перегрузки арифметических операторов:

__add__ – сложение, __sub__ – вычитание, __mul__ – умножение, __truediv__ – деление.

В классе код этих методов должен выполнять арифметические операции с давлением.

Класс включает метод convert, который помимо self принимает единицу измерения ('atm', 'bar', 'mmHg') и возвращает строку вида "101325.0 Pa = 1.0 atm", где давление переведено в указанные единицы.

Вызов экземпляра класса Pressure в нотации функции с одним аргументом должен приводить к перезаписи значения поля, в котором хранится давление, на переданное значение.

Вариант 14: Класс Power (Мощность)

Напишите класс Power по следующему описанию.

В конструкторе класса инициируется поле, содержащее мощность в ваттах, выраженное вещественным числом.

Класс включает методы перегрузки арифметических операторов:

__add__ – сложение, __sub__ – вычитание, __mul__ – умножение, __truediv__ – деление.

В классе код этих методов должен выполнять арифметические операции с мощностью.

Класс включает метод convert, который помимо self принимает единицу измерения ('kW', 'hp') и возвращает строку вида "1000.0 W = 1.0 kW", где мощность переведена в указанные единицы.

Вызов экземпляра класса Power в нотации функции с одним аргументом должен приводить к перезаписи значения поля, в котором хранится мощность, на переданное значение.

Вариант 15: Класс Frequency (Частота)

Напишите класс Frequency по следующему описанию.

В конструкторе класса инициируется поле, содержащее частоту в герцах, выраженное вещественным числом.

Класс включает методы перегрузки арифметических операторов:

__add__ – сложение, __sub__ – вычитание, __mul__ – умножение, __truediv__ – деление.

В классе код этих методов должен выполнять арифметические операции с частотой.

Класс включает метод convert, который помимо self принимает единицу измерения ('kHz', 'MHz', 'GHz') и возвращает строку вида "1000.0 Hz = 1.0 kHz", где частота переведена в указанные единицы.

Вызов экземпляра класса Frequency в нотации функции с одним аргументом должен приводить к перезаписи значения поля, в котором хранится частота, на переданное значение.

Вариант 16: Класс Resistance (Сопротивление)

Напишите класс Resistance по следующему описанию.

В конструкторе класса инициируется поле, содержащее сопротивление в омах, выраженное вещественным числом.

Класс включает методы перегрузки арифметических операторов:

__add__ – сложение, __sub__ – вычитание, __mul__ – умножение, __truediv__ – деление.

В классе код этих методов должен выполнять арифметические операции с сопротивлением.

Класс включает метод convert, который помимо self принимает единицу измерения ('kΩ', 'MΩ') и возвращает строку вида "1000.0 Ω = 1.0 kΩ", где сопротивление переведено в указанные единицы.

Вызов экземпляра класса Resistance в нотации функции с одним аргументом должен приводить к перезаписи значения поля, в котором хранится сопротивление, на переданное значение.

Вариант 17: Класс Currency (Валюта)

Напишите класс Currency по следующему описанию.

В конструкторе класса инициируется поле, содержащее количество денег, выраженное вещественным числом.

Класс включает методы перегрузки арифметических операторов:

__add__ – сложение, __sub__ – вычитание, __mul__ – умножение, __truediv__ – деление.

В классе код этих методов должен выполнять арифметические операции с количеством денег.

Класс включает метод format_currency, который помимо self принимает код валюты и символ валюты и возвращает строку вида "$100.50" или "100.50 EUR", где сумма отформатирована с символом валюты.

Вызов экземпляра класса Currency в нотации функции с одним аргументом должен приводить к перезаписи значения поля, в котором хранится количество денег, на переданное значение.

Вариант 18: Класс Percentage (Процент)

Напишите класс Percentage по следующему описанию.

В конструкторе класса инициируется поле, содержащее процентное значение, выраженное вещественным числом.

Класс включает методы перегрузки арифметических операторов:

__add__ – сложение, __sub__ – вычитание, __mul__ – умножение, __truediv__ – деление.

В классе код этих методов должен выполнять арифметические операции с процентным значением.

Класс включает метод format_percent, который возвращает строку вида "50.0%", где значение отформатировано с символом процента.

Вызов экземпляра класса Percentage в нотации функции с одним аргументом должен приводить к перезаписи значения поля, в котором хранится процент, на переданное значение.

Вариант 19: Класс Score (Баллы)

Напишите класс Score по следующему описанию.

В конструкторе класса инициируется поле, содержащее количество баллов, выраженное целым числом.

Класс включает методы перегрузки арифметических операторов:

__add__ – сложение, __sub__ – вычитание, __mul__ – умножение, __truediv__ – деление.

В классе код этих методов должен выполнять арифметические операции с баллами.

Метод __truediv__ перегружает обычное / деление с округлением до целого.

Класс включает метод calculate_grade, который помимо self принимает максимальное количество баллов и возвращает строку вида "75/100 = 75% (Оценка: 4)", где вычислен процент и оценка.

Вызов экземпляра класса Score в нотации функции с одним аргументом должен приводить к перезаписи значения поля, в котором хранятся баллы, на переданное значение.

Вариант 20: Класс Population (Население)

Напишите класс Population по следующему описанию.

В конструкторе класса инициируется поле, содержащее количество населения, выраженное целым числом.

Класс включает методы перегрузки арифметических операторов:

__add__ – сложение, __sub__ – вычитание, __mul__ – умножение, __truediv__ – деление.

В классе код этих методов должен выполнять арифметические операции с населением.

Метод __truediv__ перегружает обычное / деление с округлением до целого.

Класс включает метод calculate_density, который помимо self принимает площадь территории и возвращает строку вида "Население 1000000 на площади 1000 км² = 1000 чел/км²", где вычислена плотность населения.

Вызов экземпляра класса Population в нотации функции с одним аргументом должен приводить к перезаписи значения поля, в котором хранится население, на переданное значение.

Вариант 21: Класс Memory (Память)

Напишите класс Memory по следующему описанию.

В конструкторе класса инициируется поле, содержащее объем памяти в байтах, выраженное целым числом.

Класс включает методы перегрузки арифметических операторов:

__add__ – сложение, __sub__ – вычитание, __mul__ – умножение, __truediv__ – деление.

В классе код этих методов должен выполнять арифметические операции с объемом памяти.

Метод __truediv__ перегружает обычное / деление с округлением до целого.

Класс включает метод format_memory, который возвращает строку вида "1.5 GB", где объем автоматически конвертируется в подходящие единицы (B, KB, MB, GB, TB).

Вызов экземпляра класса Memory в нотации функции с одним аргументом должен приводить к перезаписи значения поля, в котором хранится объем памяти, на переданное значение.

Вариант 22: Класс Brightness (Яркость)

Напишите класс Brightness по следующему описанию.

В конструкторе класса инициируется поле, содержащее значение яркости, выраженное целым числом (0-100).

Класс включает методы перегрузки арифметических операторов:

__add__ – сложение, __sub__ – вычитание, __mul__ – умножение, __truediv__ – деление.

В классе код этих методов должен выполнять арифметические операции с яркостью.

Значение яркости должно оставаться в диапазоне 0-100 после операций.

Класс включает метод get_level, который возвращает строку вида "Яркость: 75% (Высокая)", где кроме процентного значения указывается уровень яркости (Низкая, Средняя, Высокая).

Вызов экземпляра класса Brightness в нотации функции с одним аргументом должен приводить к перезаписи значения поля, в котором хранится яркость, на переданное значение.

Вариант 23: Класс Concentration (Концентрация)

Напишите класс Concentration по следующему описанию.

В конструкторе класса инициируется поле, содержащее концентрацию в молях на литр, выраженное вещественным числом.

Класс включает методы перегрузки арифметических операторов:

__add__ – сложение, __sub__ – вычитание, __mul__ – умножение, __truediv__ – деление.

В классе код этих методов должен выполнять арифметические операции с концентрацией.

Класс включает метод convert, который помимо self принимает единицу измерения ('mmol/L', 'g/L') и возвращает строку вида "1.0 mol/L = 1000.0 mmol/L", где концентрация переведена в указанные единицы.

Вызов экземпляра класса Concentration в нотации функции с одним аргументом должен приводить к перезаписи значения поля, в котором хранится концентрация, на переданное значение.

Вариант 24: Класс Voltage (Напряжение)

Напишите класс Voltage по следующему описанию.

В конструкторе класса инициируется поле, содержащее напряжение в вольтах, выраженное вещественным числом.

Класс включает методы перегрузки арифметических операторов:

__add__ – сложение, __sub__ – вычитание, __mul__ – умножение, __truediv__ – деление.

В классе код этих методов должен выполнять арифметические операции с напряжением.

Класс включает метод calculate_power, который помимо self принимает силу тока и возвращает строку вида "Напряжение 220V при токе 5A = 1100W мощности", где вычислена мощность по формуле P = U*I.

Вызов экземпляра класса Voltage в нотации функции с одним аргументом должен приводить к перезаписи значения поля, в котором хранится напряжение, на переданное значение.

Вариант 25: Класс Resolution (Разрешение)

Напишите класс Resolution по следующему описанию.

В конструкторе класса инициируется поле, содержащее общее количество пикселей, выраженное целым числом.

Класс включает методы перегрузки арифметических операторов:

__add__ – сложение, __sub__ – вычитание, __mul__ – умножение, __truediv__ – деление.

В классе код этих методов должен выполнять арифметические операции с количеством пикселей.

Метод __truediv__ перегружает обычное / деление с округлением до целого.

Класс включает метод get_resolution, который помимо self принимает соотношение сторон (например, 16/9) и возвращает строку вида "2073600 пикселей = 1920x1080 (16:9)", где вычислены ширина и высота экрана исходя из общего количества пикселей и соотношения сторон.

Вызов экземпляра класса Resolution в нотации функции с одним аргументом должен приводить к перезаписи значения поля, в котором хранится количество пикселей, на переданное значение.